

Usando a linguagem de programação 5, crie o software que realiza as 5 funções a seguir. As funções devem aparecer num menu, e o usuário poderá escolher qual deseja realizar.

O trabalho deve ser realizado em grupos com 5 pessoas. Grupos individuais ou com quantidades menores que essa não serão aceitas. Os alunos ficarão com nota ZERO (0).

Depois de feitas as funções, elas devem ser implementadas em pseudocódigo, depois a complexidade de cada linha de comando deve ser exibida, e ainda se deve calcular a expressão de complexidade da função toda e o Big O dela.

Para cada função, ainda é preciso fazer um cálculo do tempo gasto considerando-se a entrada fornecida no final de cada questão (na parte descrita como “Cálculo de Tempo”) e considerando-se que um computador realiza 10^8 instruções por segundo.

Só 1 dos membros da equipe deve mandar o código em C do programa (pode ser o arquivo .cpp mesmo) e um arquivo de texto com os pseudocódigos e cálculos de complexidade e tempo;

O membro que enviar os arquivos deverá informar (no programa e no arquivo de texto) os nomes dos 5 membros do grupo. Se alguma pessoa não tiver seu nome nesta lista até a hora da correção, a pessoa ficará com nota zero (ou seja, não aceito ninguém sendo adicionado aos grupos depois que os arquivos forem enviados).

São 5 funções que devem apresentar a complexidade, mas eventualmente vocês precisarão criar mais algumas pra poder testar as 5 funções solicitadas. Essas funções extras não serão julgadas, logo não precisa fazer a complexidade delas.

Aqui neste documento, depois das 5 questões, vocês ainda têm mais algumas orientações que precisam ser seguidas (no tópico marcado como ATENÇÃO).

Função 1: Contagem de Ocorrências Distintas

- **Descrição:** Escreva uma função que receba um vetor de inteiros de tamanho n e uma lista de k elementos a serem buscados (vetor de tamanho k). A função deve percorrer a lista de k elementos e, para cada um, contar quantas vezes ele aparece no vetor principal de n elementos, retornando a soma da quantidade de aparições de cada elemento no vetor de buscados (Exemplo, considere que o vetor de buscados tinha 2 elementos. O primeiro apareceu 3 vezes e o segundo apareceu 5 vezes no vetor de inteiros, então a função deve retornar 8, que é $3+5$).
- **Cálculo de Tempo:** Considere $n = 50.000$ e $k = 4.000$.

Função 2: Análise de Pares em Matriz Triangular

- **Descrição:** Escreva uma função que receba uma matriz de inteiros com n linhas e n colunas. A função deve **testar** todos os elementos da metade superior da matriz (os que estão acima da diagonal principal) com os seus opostos presentes na metade inferior (os que estão abaixo da diagonal principal). Ou seja, um contador deve ser incrementado toda vez que $A[i][j] + A[j][i]$ for um valor múltiplo de 5. Retorne o total contado. A diagonal principal também deve ser avaliada.
- **Dica:** o for interno deve ser executado exclusivamente quando o elemento for da diagonal principal, ou da metade superior da matriz. Isso vai promover uma quantidade de repetições que não é regular, mas que também já vimos em sala de aula anteriormente, então estudem o caso e coloquem a complexidade correta no for interno.
- **Cálculo de Tempo:** Considere $n = 500$.

Considerando uma matriz 4x4, as células em azul são a diagonal principal, as células em amarelo são a metade inferior, e aquelas em laranja são da metade superior.

Função 3: Comparação de Matrizes Tridimensionais

- **Descrição:** Escreva uma função que receba dois arranjos tridimensionais de inteiros, A e B, ambos com dimensões $n \times n \times n$. A função deve:
 1. Percorrer o arranjo A completamente para calcular a soma de todos os seus elementos.
 2. Percorrer o arranjo B completamente para calcular a soma de todos os seus elementos.
 3. Comparar as duas somas finais e retornar 1 se a soma de A for maior ou igual à de B, e 0 caso contrário.
- **Cálculo de Tempo:** Considere $n = 300$.

Função 4 (Refeita): Análise de Casos Assimétricos no Condicional

- **Descrição:** Escreva uma função `processar_vetor` que receba um vetor de inteiros de tamanho n . A função percorre o vetor com um laço principal e verifica cada elemento $V[i]$:
 - **Se o elemento for PAR:** A função apenas soma o valor do $V[i]$ atual a uma variável que guarda o somatório.
 - **Se o elemento for ÍMPAR:** A função calcula o fatorial daquele valor e o resultado é somado à variável que guarda o somatório.
 - A função retorna o somatório ao final.

- **Cálculo de Tempo:** Considere $n = 50.000$ no pior caso.

Função 5: Contagem de Elementos Presentes em Vetor Ordenado

- **Descrição:** Implemente primeiro uma função auxiliar de **Busca Binária** em um vetor (que retorna 1 se o elemento for encontrado e 0 caso contrário). Em seguida, implemente a função que recebe um vetor A não ordenado de tamanho n e um vetor B ordenado de tamanho n . Para cada elemento do vetor A, a função deve realizar uma busca binária no vetor B. A função deve retornar o total de elementos do vetor A que foram encontrados no vetor B.
- **Cálculo de Tempo:** Considere $n = 10.000.000$.

ATENÇÃO:

- 01) Adicionalmente, crie uma função que pergunta quantas linhas e quantas colunas se deseja que um arranjo tenha. A função cria o arranjo (usando Matriz Dinâmica) e preenche todo esse arranjo com valores aleatórios. No menu principal do programa, sempre que uma das 5 funções for escolhida pelo usuário, o programa deve perguntar se ele quer preencher o arranjo manualmente ou se quer que essa função preencha os valores automaticamente com valores randômicos.
- 02) Em cada chamada de função, é preciso imprimir na tela o arranjo que foi criado, pra que se possa conferir se a função está realmente executando o que foi criada para fazer, e só depois ela mostra o resultado de sua operação.
- 03) Estudem sobre Matrizes Estáticas ou Alacadas Dinamicamente. Desde a versão 99 do C (a C99), a linguagem passou a suportar a suportar VLA (Variable Length Arrays / Arranjos de Tamanho Variável), essa é a maneira mais limpa e direta para matrizes onde as linhas e colunas são passadas dinamicamente. A única regra obrigatória é que a dimensão n deve ser declarada no parâmetro antes da matriz:

```
void exemplo(int n, int M[n][n]) {...}
```